

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH

BÀI TẬP LỚN – DEMO CÓ GIAO DIỆN

ĐƯỜNG ĐI AN TOÀN

Thực quan hóa thuật toán Quy hoạch động đếm đường đi trên lưới

Tên môn học: Phân tích & Thiết kế thuật toán
Mã lớp: CS112.2026.TX
Giảng viên: Huỳnh Thị Thanh Thương
Sinh viên: Đinh Văn Huy – 25730032
Hình thức: Cá nhân
Bài toán chọn: “Đường đi an toàn” (WeCode – nhóm Dynamic Programming)

Contents

1	Yêu cầu của bài tập lớn	2
2	Phát biểu bài toán	2
3	Thiết kế thuật toán	2
3.1	Vì sao không vét cạn?	2
3.2	Trạng thái và công thức truy hồi	3
3.3	Chứng minh tính đúng đắn	3
3.4	Mã giả	3
3.5	Độ phức tạp	3
3.6	Ví dụ chạy tay (ví dụ trong đề)	3
4	Kiểm chứng lời giải	4
5	Sản phẩm: giao diện trực quan hóa	4
5.1	Công nghệ và cách chạy	4
5.2	Các chức năng chính	4
5.3	Video demo	8
6	Quy trình thực hiện và cách sử dụng AI	8
6.1	Công cụ	8
6.2	Các bước và prompt đã dùng	9
6.3	Phân định: phần nào do AI, phần nào do em	9
6.4	Nhận xét về việc dùng AI	10
7	Đánh giá và hướng phát triển	10
A	Phụ lục: lời giải nộp WeCode (Python)	10
B	Phụ lục: cấu trúc thư mục nộp	11

1. Yêu cầu của bài tập lớn

Theo yêu cầu cô giao ở Buổi 8 (đầu buổi, khoảng phút 12–22; nhắc lại cuối buổi), sinh viên chọn một bài toán đã học hoặc đã làm trên WeCode và xây dựng một **demo có giao diện**: có ô nhập dữ liệu, khung hiển thị kết quả, minh họa trực quan quá trình giải; được phép dùng AI. Sản phẩm nộp gồm ba phần:

1. **Source code** của sản phẩm (thư mục `app/`, `solution/`, `tests/`).
2. **Video** quay quá trình chạy thử (`video/demo_HoaiMy.mp4`).
3. **Report** (tài liệu này) tường thuật lại toàn bộ quy trình: tìm hiểu, dùng AI nào, prompt ra sao, thuật toán do ai đề xuất và kiểm chứng thế nào.

Em chọn bài **Đường đi an toàn** vì bài có lời giải quy hoạch động (QHD) ngắn gọn nhưng rất phù hợp để minh họa: mỗi ô của bảng là một bài toán con, và việc “điền bảng” có thể diễn ra trực tiếp trên bản đồ. Mục tiêu đặt ra không chỉ là cho ra đáp án, mà là giúp **người mới học hiểu được vì sao công thức truy hồi đúng**.

2. Phát biểu bài toán

Đề bài (tóm tắt)

Bản đồ làng đại học kích thước $n \times n$; một số ô có chốt cảnh sát giao thông. Tấn Lưu đi từ KTX khu B ở ô $[1, 1]$ (góc trái trên) tới UIT ở ô $[n, n]$ (góc phải dưới), mỗi bước chỉ được **sang phải** hoặc **xuống dưới**. Đếm số con đường không đi qua chốt nào, lấy dư cho $10^9 + 7$.

Input

- n : số nguyên, $1 \leq n \leq 1000$ – kích thước bản đồ.
- $A = (a_{ij})_{n \times n}$, $a_{ij} \in \{., *\}$ – bản đồ; $a_{ij} = *$ nghĩa là ô (i, j) có chốt, $a_{ij} = .$ là ô trống. Cho dưới dạng n dòng, mỗi dòng đúng n ký tự.

Output

- R : số nguyên, $0 \leq R < 10^9 + 7$, với $R = |\mathcal{P}| \bmod (10^9 + 7)$, trong đó $\mathcal{P}$ là tập các dãy ô $(1, 1) = c_0, c_1, \dots, c_{2n-2} = (n, n)$ sao cho mỗi c_{k+1} là ô bên phải hoặc ô phía dưới của c_k , và $a_{c_k} = .$ với mọi k .

Ví dụ trong đề (hình minh họa). Với $n = 4$, bản đồ `.... / .*.. / ...* / *...` thì đáp án là **3**.

3. Thiết kế thuật toán

3.1 Vì sao không vét cạn?

Khi không có chốt, mỗi đường gồm $n - 1$ bước xuống và $n - 1$ bước sang phải theo thứ tự tùy ý, nên có $\binom{2n-2}{n-1}$ đường. Với $n = 1000$ con số này cỡ 10^{598} – liệt kê từng đường là bất khả thi. Tuy nhiên rất nhiều đường *dùng chung đoạn đầu*: số cách đi tới một ô chỉ phụ thuộc vào ô đó chứ không phụ thuộc vào phần đường còn lại. Đây chính là tính chất **bài toán con gối nhau** – dấu hiệu dùng quy hoạch động.

3.2 Trạng thái và công thức truy hồi

Gọi $dp[i][j]$ là số đường an toàn từ $(1, 1)$ tới (i, j) . Quy ước $dp[i][j] = 0$ nếu $i = 0$ hoặc $j = 0$ (ngoài bản đồ).

$$dp[i][j] = \begin{cases} 0 & \text{nếu } a_{ij} = *, \\ 1 & \text{nếu } (i, j) = (1, 1) \text{ và } a_{11} = ., \\ (dp[i-1][j] + dp[i][j-1]) \bmod (10^9 + 7) & \text{ngược lại.} \end{cases}$$

Đáp án: $dp[n][n]$.

3.3 Chứng minh tính đúng đắn

Xét ô trống $(i, j) \neq (1, 1)$. Mọi đường an toàn tới (i, j) có *bước cuối* hoặc là bước xuống từ $(i-1, j)$, hoặc là bước sang phải từ $(i, j-1)$ – không còn khả năng nào khác vì chỉ được đi phải/xuống. Hai nhóm này **rời nhau** (bước cuối khác nhau) và **phủ hết** mọi đường. Bỏ bước cuối của một đường thuộc nhóm thứ nhất ta được đúng một đường an toàn tới $(i-1, j)$ và ngược lại (song ánh), tương tự cho nhóm thứ hai. Theo quy tắc cộng, số đường tới (i, j) bằng $dp[i-1][j] + dp[i][j-1]$. Ô có chốt không thể nằm trên đường an toàn nên $dp = 0$. Trường hợp cơ sở $(1, 1)$: có đúng một đường (đường rỗng). Bằng quy nạp theo thứ tự duyệt (hàng tăng dần, trong mỗi hàng cột tăng dần) – khi tính (i, j) thì $(i-1, j)$ và $(i, j-1)$ đã được tính đúng – ta có mọi $dp[i][j]$ đều đúng.

Việc lấy dư sau mỗi phép cộng là hợp lệ vì $(x+y) \bmod p = ((x \bmod p) + (y \bmod p)) \bmod p$.

3.4 Mã giả

```

1 SafePaths(n, A):
2   P <- 10^9 + 7
3   dp[1..n] <- [1, 0, 0, ..., 0]
4   for i <- 1 to n:
5     for j <- 1 to n:
6       if A[i][j] = '*': dp[j] <- 0
7       else if j > 1: dp[j] <- (dp[j] + dp[j-1]) mod P
8   return dp[n]
```

Nén bộ nhớ. Hàng i chỉ phụ thuộc hàng $i-1$. Dùng một mảng $dp[1..n]$: ngay trước khi cập nhật, $dp[j]$ vẫn giữ giá trị của ô phía trên $(i-1, j)$, còn $dp[j-1]$ vừa được cập nhật nên là ô bên trái $(i, j-1)$. Với $j=1$ chỉ có ô phía trên nên giữ nguyên $dp[1]$ (trừ khi gặp chốt). Ô $(1, 1)$ được khởi tạo bằng 1 trước vòng lặp; nếu $(1, 1)$ là chốt, lệnh ở dòng 6 đưa nó về 0 – đúng.

3.5 Độ phức tạp

Mỗi ô được xử lý đúng một lần với $O(1)$ phép tính: $T(n) = \sum_{i=1}^n \sum_{j=1}^n c = cn^2 \in \Theta(n^2)$. Với $n = 1000$ là 10^6 phép tính. Bộ nhớ phụ $\Theta(n)$ cho mảng dp (không tính bộ nhớ đọc input – lời giải đọc và xử lý từng dòng nên cũng chỉ cần $\Theta(n)$).

3.6 Ví dụ chạy tay (ví dụ trong đề)

dp	1	2	3	4
1	1	1	1	1
2	1	0(*)	1	2
3	1	1	2	0(*)
4	0(*)	1	3	3

Chẳng hạn $dp[4][4] = dp[3][4] + dp[4][3] = 0 + 3 = 3$ – khớp đáp án.

4. Kiểm chứng lời giải

Thuật toán được AI đề xuất (xem mục 6), vì vậy em không tin ngay mà kiểm chứng bằng ba cách:

1. **Chạy tay** ví dụ trong đề (bảng ở trên) – ra 3, đúng.
2. **Đối chiếu với vết cặn** (`tests/run_tests.py`): một hàm liệt kê từng đường đi bằng ngăn xếp (không nhớ, không dùng công thức QHĐ) làm “đáp án chuẩn”. Chạy trên 307 bản đồ: ví dụ trong đề, 6 trường hợp biên ($n = 1$, chót ở KTX, chót ở UIT, đường chéo bị chặn,...) và 300 bản đồ ngẫu nhiên $n \leq 6$ với mật độ chót 0%–70%. Cả lời giải Python lẫn lời JavaScript của giao diện web đều cho kết quả trùng khớp **307/307**.
3. **Test lớn và phép lấy dư**: với $n = 1000$ không có chót, đáp án phải là $\binom{1998}{999} \bmod (10^9 + 7) = 965\,601\,742$ (tính độc lập bằng `math.comb` của Python); cả hai bản cài đặt đều ra đúng số này. Thời gian: Python $\approx 0,14$ s, JavaScript $\approx 0,06$ s (đã gồm khởi động trình thông dịch). Ngoài ra bộ test còn kiểm tra 7 input sai (rỗng, n không phải số, $n = 0$, $n = 1001$, thiếu dòng, ký tự lạ, dòng dài sai) – giao diện từ chối cả 7.

[Cần sinh viên xác nhận trước khi nộp] Lời giải Python ở Phụ lục là bản em đã nộp trên WeCode; trạng thái chấm trên WeCode: _____ (ví dụ: Accepted 100/100).

5. Sản phẩm: giao diện trực quan hóa

5.1 Công nghệ và cách chạy

Ứng dụng web tĩnh: HTML + CSS + JavaScript thuần, không cần cài đặt hay build. Lời thuật toán nằm riêng trong `app/core.js` để dùng chung cho trình duyệt và cho bộ test bằng Node.js.

- Chạy: mở `app/index.html` bằng trình duyệt, hoặc `python3 -m http.server -d app` rồi vào `http://localhost:8000`.
- Kiểm thử: `python3 tests/run_tests.py`.
- **Bản online**: <https://vanhuydotcom.github.io/cs112-duong-di-an-toan/> – mã nguồn: <https://github.com/vanhuydotcom/cs112-duong-di-an-toan>.

5.2 Các chức năng chính

1. **Nhập dữ liệu** đúng định dạng WeCode (ô văn bản); 4 ví dụ có sẵn; **sinh bản đồ ngẫu nhiên** với n và mật độ chót tùy chọn; **bấm trực tiếp lên lưới** để thêm/bỏ chót.
2. **Kiểm tra input đầy đủ**: n không phải số nguyên, n ngoài $[1, 1000]$, sai số dòng, dòng sai độ dài, ký tự khác `./*` – báo lỗi kèm số dòng, hàng, cột.
3. **Minh họa từng bước**: điền bảng dp theo đúng thứ tự duyệt; ô đang tính tô vàng, ô phía trên tô tím, ô bên trái tô xanh, kèm mũi tên; khung giải thích ghi rõ công thức với số cụ thể, ví dụ $dp[3][2] = dp[2][2] + dp[3][1] = 0 + 1 = 1$; mã giả tô sáng dòng đang thực thi; nhật ký các phép tính.
4. **Điều khiển**: chạy/dừng, tiến/lùi từng bước, về đầu/tới cuối, chỉnh tốc độ; phím tắt Space, $\leftarrow$, $\rightarrow$, Home, End.
5. **Hai chế độ xem**: bảng 2 chiều $dp[i][j]$ (dễ hiểu) và mảng 1 chiều $dp[j]$ – đúng như code nộp WeCode, cho thấy vì sao nén bộ nhớ vẫn đúng.

- Kết quả:** đáp án mod $10^9 + 7$, *số đường thật* (số nguyên lớn, BigInt) để thấy vì sao phải lấy dư, thời gian chạy; vẽ ngẫu nhiên một đường an toàn; làm mờ các ô không nằm trên đường an toàn nào.
- Lưới lớn** ($n > 16$, tới 1000): không hoạt hình từng ô mà hiển thị bản đồ nhiệt (màu theo log số đường tới mỗi ô) cùng kết quả.
- Song ngữ** Việt/Anh, giao diện sáng/tối, dùng được trên điện thoại.

Đường đi an toàn

Trực quan hóa Quy hoạch động · CS112 – Phân tích & Thiết kế thuật toán · UIT

Tiếng Việt

English

Auto

1 Bài toán

Tấn Lưu chở ba từ **KTX khu B** (góc trái trên, ô $[1,1]$) về **UIT** (góc phải dưới, ô $[n,n]$) trên bản đồ $n \times n$. Một số ô có **chốt cảnh sát giao thông** (ký tự *).

Mỗi bước chỉ được đi **sang phải** hoặc **xuống dưới**. Hãy đếm số con đường **không đi qua chốt nào**, lấy dư cho 10^9+7 .

Input dòng 1: n ($1 \leq n \leq 1000$); dòng sau: mỗi dòng n ký tự (ô trống) hoặc * (chốt).

Output một số nguyên – số đường đi an toàn mod 10^9+7 .

2 Nhập dữ liệu

Ví dụ trong đề

Không có chốt

Bị chặn hết

Chốt ngay KTX

```
4
....
.*..
...*
*...
```

Sinh ngẫu nhiên: $n =$

mật độ chốt

☒ giữ trống KTX & UIT

Mẹo: bấm vào một ô trên lưới để bật/tắt chốt CSGT.

Hợp lệ: lưới 4×4 , 3 chốt. Bấm ▶ Chạy để xem từng bước.

3 Quy hoạch động từng bước

Bảng $dp[i][j]$ (2 chiều)

Mảng $dp[i][j]$ (code nộp WeCode)

	1	2	3	4
1				
2				
3				
4				

Sẵn sàng

Bấm ▶ Chạy hoặc ▶ để di chuyển theo thứ tự từng hàng, trái → phải. Khi tới ô (i,j) , mọi ô phía trên và bên trái đã có kết quả.

Mã giả

```
1 for i = 1..n:
2   for j = 1..n:
3     if map[i][j] == '*': dp[i][j] = 0
4     elif (i,j) == (1,1): dp[i][j] = 1
5     else: dp[i][j] = (dp[i-1][j] + dp[i][j-1]) mod P
6   print dp[n][n]
```

☐ ô đang tính
 ☐ ô phía trên
 ☐ ô bên trái
 ☐ chốt CSGT
 ☐ một đường an toàn

Phím tắt: Space chạy/dừng · ← lùi/tiến · Home End đầu/cuối

4 Kết quả

Output (mod 10^9+7)

Số đường thật (chưa lấy dư)

Kích thước · số chốt · thời gian

3

3

$4 \times 4 \cdot 3 \cdot 0.00$ ms

Vẽ một đường an toàn ngẫu nhiên

Làm mờ ô không nằm trên đường an toàn

5 Vì sao thuật toán đúng & nhanh?

- Định nghĩa trạng thái.** $dp[i][j]$ = số đường an toàn từ KTX $(1,1)$ tới ô (i,j) .
- Công thức truy hồi.** Mọi đường tới ô (i,j) có bước cuối là từ $(i-1,j)$ hoặc từ $(i,j-1)$; hai nhóm rời nhau và phủ hết, nên $dp[i][j] = 0$ nếu ô (i,j) là chốt, ngược lại $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ (ô ngoài bản đồ = 0).
- Cơ sở.** $dp[1][1] = 1$ nếu KTX không có chốt. **Đáp án** $dp[n][n] \bmod (10^9+7)$. Lấy dư sau mỗi phép cộng là đúng vì $(a+b) \bmod p = ((a \bmod p) + (b \bmod p)) \bmod p$.
- Thứ tự tính.** Duyệt hàng trên → dưới, cột trái → phải: khi tính ô (i,j) thì ô $(i-1,j)$ và ô $(i,j-1)$ đã xong.
- Độ phức tạp.** Mỗi ô làm $O(1)$ việc $\Rightarrow O(n^2)$ thời gian ($n = 1000: 10^6$ phép tính). Chỉ cần hàng trước $\Rightarrow$ nên cần $O(n)$ bộ nhớ (mảng $dp[j]$).
- Số vết vét cạn.** Không có chốt thì số đường là $C(2n-2, n-1)$ – với $n = 1000$ lớn cỡ 10^{300} , liệt kê từng đường là bất khả thi. Đây là ví dụ kinh điển của **bài toán con gối nhau**: QHĐ lưu lại để không tính lại.

Đinh Văn Huy – 25730032 · CS112.2026.TX · Bài tập lớn: demo có giao diện · Lời giải chạy $O(n^2)$ thời gian, $O(n)$ bộ nhớ.

Figure 1: Toàn cảnh giao diện: đề bài, nhập dữ liệu, minh họa, kết quả, phân tích.

3 Quy hoạch động từng bước

Bảng $dp[i][j]$ (2 chiều) Mảng $dp[j]$ (code nộp WeCode)

	1	2	3	4
1	1	1	1	1
2	1	0		
3				
4				

■ ô đang tính ■ ô phía trên ■ ô bên trái ■ chốt CSGT ■ một đường an toàn

Phím tắt: Space chạy/dừng ← lùi/tiến Home End đầu/cuối

Ô (2,2) có chốt

Không đường an toàn nào được đi qua ô này $\Rightarrow dp = 0$. Ô phía dưới/bên phải sẽ không nhận được gì từ đây.

$dp[2][2] = 0$

Mã giả

```

1 for i ← 1..n:
2   for j ← 1..n:
3     if map[i][j] = '*': dp[i][j] ← 0
4     elif (i,j) = (1,1): dp[i][j] ← 1
5     else: dp[i][j] ← (dp[i-1][j] + dp[i][j-1]) mod P
6   print dp[n][n]
  
```

1. $dp[1][1] = 1$
 2. $dp[1][2] = 0 + 1 = 1$
 3. $dp[1][3] = 0 + 1 = 1$
 4. $dp[1][4] = 0 + 1 = 1$
 5. $dp[2][1] = 1 + 0 = 1$
 6. $dp[2][2] = 0$ (chốt)

Figure 2: Bước tính một ô: cộng ô phía trên (tím) và ô bên trái (xanh); mã giả tô sáng dòng 5.

3 Quy hoạch động từng bước

Bảng $dp[i][j]$ (2 chiều) Mảng $dp[j]$ (code nộp WeCode)

	1	2	3	4
1	1	1	1	1
2	1	0	1	2
3	1	1		
4				

■ ô đang tính ■ ô phía trên ■ ô bên trái ■ chốt CSGT ■ một đường an toàn

Phím tắt: Space chạy/dừng ← lùi/tiến Home End đầu/cuối

Ô (3,2)

Bước cuối cùng để tới ô này chỉ có thể đến từ trên xuống hoặc từ trái sang. Hai nhóm đường này không trùng nhau $\Rightarrow$ cộng lại.

$dp[3][2] = dp[2][2] + dp[3][1] = 0 + 1 = 1$

Code thực tế chỉ giữ một hàng $dp[0..n-1]$. Trước khi cập nhật, $dp[j]$ còn là giá trị của ô phía trên; $dp[j-1]$ vừa được cập nhật nên là ô bên trái.

1 $dp[0]$ 1 $dp[1]$ 1 $dp[2]$ 2 $dp[3]$

Mã giả

```

1 dp ← [1, 0, 0, ..., 0]      # length n
2 for each row in map:
3   for j in 0..n-1:
4     if row[j] = '*': dp[j] ← 0
5     elif j > 0: dp[j] ← (dp[j] + dp[j-1]) mod P
6   print dp[n-1]
  
```

3. $dp[1][3] = 0 + 1 = 1$
 4. $dp[1][4] = 0 + 1 = 1$
 5. $dp[2][1] = 1 + 0 = 1$
 6. $dp[2][2] = 0$ (chốt)
 7. $dp[2][3] = 1 + 0 = 1$
 8. $dp[2][4] = 1 + 1 = 2$
 9. $dp[3][1] = 1 + 0 = 1$
 10. $dp[3][2] = 0 + 1 = 1$

Figure 3: Chế độ mảng 1 chiều $dp[j]$ – minh họa đúng code nộp WeCode.

3 Quy hoạch động từng bước

Bảng dp[i][j] (2 chiều)

Mảng dp[j] (code nộp WeCode)

	1	2	3	4	5	6
1	🏠 1	1	1	1	1	1
2	1	🚗 0	1	2	3	4
3	1	1	2	🚗 0	3	7
4	🚗 0	1	3	3	6	13
5	0	1	🚗 0	3	9	22
6	0	1	1	4	13	🏠 35

🟡 ô đang tính

🟠 ô phía trên

🟢 ô bên trái

🚗 chất CSGT

🟩 một đường an toàn

⏮ ⏪ ▶ Chạy ⏩ ⏭

Tốc độ

Phím tắt: Space chạy/dừng · ⇐ ⇨ lùi/tiến · Home End đầu/cuối

Ô (6,6)

Bước cuối cùng để tới ô này chỉ có thể đến từ trên xuống hoặc từ trái sang. Hai nhóm đường này không trùng nhau ⇒ cộng lại.

$dp[6][6] = dp[5][6] + dp[6][5] = 22 + 13 = 35$

Hoàn tất! Đáp án là giá trị ở ô UIT (6,6). → 35

Mã giả

```

1 for i ← 1..n:
2   for j ← 1..n:
3     if map[i][j] = '*': dp[i][j] ← 0
4     elif (i,j) = (1,1): dp[i][j] ← 1
5     else: dp[i][j] ← (dp[i-1][j] + dp[i][j-1]) mod P
6 print dp[n][n]
```

29. $dp[5][5] = 6 + 3 = 9$

30. $dp[5][6] = 13 + 9 = 22$

31. $dp[6][1] = 0 + 0 = 0$

32. $dp[6][2] = 1 + 0 = 1$

33. $dp[6][3] = 0 + 1 = 1$

34. $dp[6][4] = 3 + 1 = 4$

35. $dp[6][5] = 9 + 4 = 13$

36. $dp[6][6] = 22 + 13 = 35$

Figure 4: Bản đồ 6×6 (35 đường): một đường an toàn ngẫu nhiên (viền xanh lá), các ô không nằm trên đường nào bị làm mờ.

2

Nhập dữ liệu

Ví dụ trong đề

Không có chốt

Bị chặn hết

Chốt ngay KTX

6

.....
.*.....
...*..
*.....
..*.....
.....

Sinh ngẫu nhiên: n =

7

mật độ chốt

28%

☒ giữ trống KTX & UIT

☐ Sinh

☒ Kiểm tra & ngp

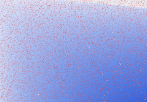
Mẹo: bấm vào một ô trên lưới để bật/tắt chốt CSGT.

Input chưa hợp lệ:

- Cần đúng 3 dòng bản đồ nhưng có 2 dòng.
- Dòng 2, hàng 1, cột 3: ký tự "x" không hợp lệ (chỉ dùng . hoặc *).
- Dòng 3 (hàng 2 của bản đồ) có 4 ký tự, cần đúng 3.

📌 Quy hoạch động tăng bước

Bảng dp[i][j] (2 chiều)



Mảng dp[i][j] (còn nạp WeCode)

Hoàn tất!

Đáp án là giá trị ở ô UT (1000,1000).
dp[1000][1000] mod P = 590188668

Mô giả

```

1 for i = 1..N;
2   for j = 1..N;
3     if dp[i][j] < '*'; dp[i][j] = 0
4     else dp[i,j] = (d[i,j] - dp[i-1][j]) + 1
5   else: dp[i][j] = (dp[i-1][j] + dp[i][j-1]) mod P
6 print dp[N][N]
```

Phạm Anh: bscs/copy/chung - ... - lữ/thức - đươc /xã/đầu/bước

Figure 5: Trái: báo lỗi input kèm vị trí. Phải: lưới 1000×1000 hiển thị bằng bản đồ nhiệt.

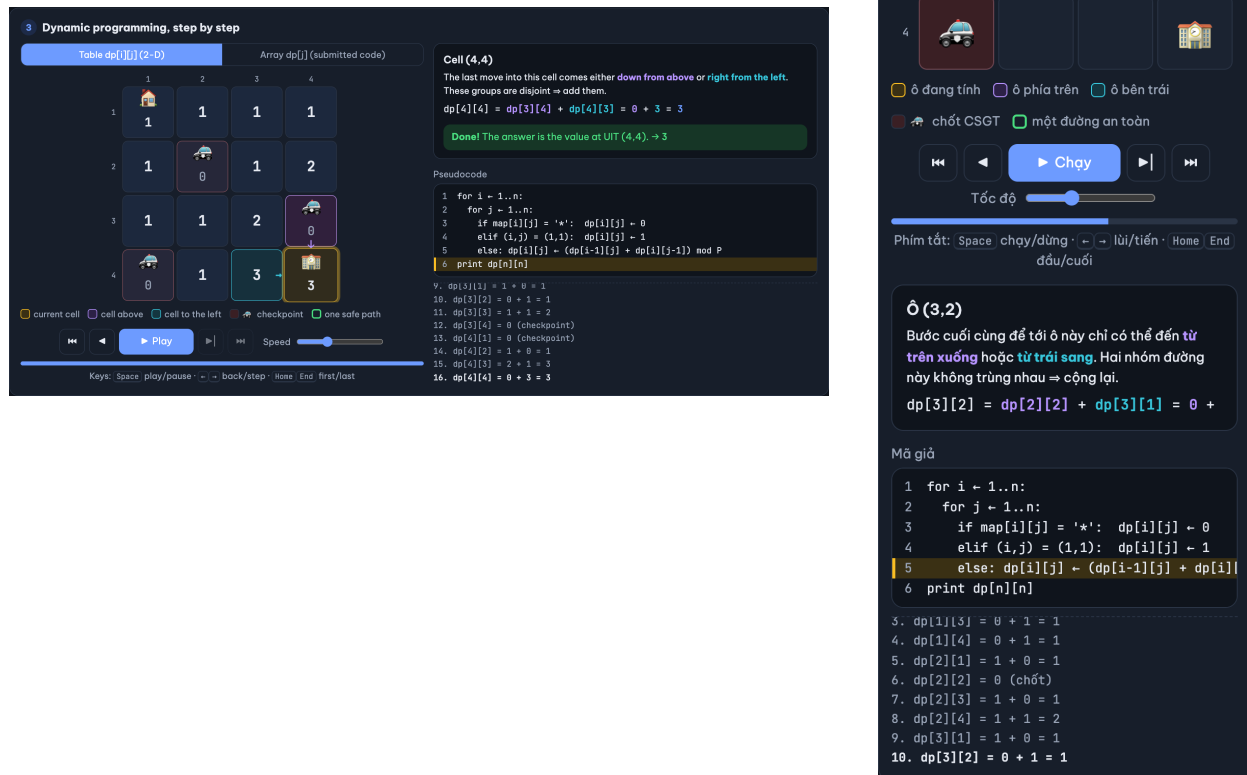


Figure 6: Giao diện tiếng Anh, nền tối (trái) và trên điện thoại (phải).

5.3 Video demo

Video `video/demo_HoaiMy.mp4` hoặc `demo_NamMinh.mp4` (khoảng 3 phút, có thuyết minh tiếng Việt và phụ đề) được **tạo tự động** bằng script `video/make_video.py`: Playwright điều khiển trình duyệt Chrome theo kịch bản 16 cảnh và ghi hình; lời thuyết minh được đọc bằng giọng neural tiếng Việt của Microsoft qua thư viện `edge-tts` (bản đầu dùng giọng “Linh” của macOS nhưng nghe không tự nhiên nên đã thay); `ffmpeg` ghép hình và tiếng. Cách làm này giúp video có thể tạo lại y hệt bất cứ lúc nào sau khi sửa giao diện.

6. Quy trình thực hiện và cách sử dụng AI

6.1 Công cụ

- **AI:** Claude Code (mô hình Claude Opus 5.5 của Anthropic), chạy trong ứng dụng Claude desktop, có quyền đọc/ghi file trong thư mục môn học và chạy lệnh trên máy.
- **Công cụ khác:** Python 3, Node.js, Playwright + Google Chrome (chụp ảnh, quay video), `ffmpeg`, `edge-tts` (giọng đọc), Tectonic, GitHub Pages (đăng demo) (biên dịch LaTeX).

6.2 Các bước và prompt đã dùng

Toàn bộ quá trình diễn ra trong **một phiên làm việc** với Claude Code. Dưới đây là các prompt của em (giữ nguyên văn, chỉ rút gọn phần dài) và việc AI đã làm sau mỗi prompt.

Prompt của em	AI đã làm
<i>“check tài liệu trong docs đọc transcript và cho tôi biết ở buổi mấy, mốc thời gian nào cô yêu cầu làm bài tập lớn, nội dung yêu cầu là gì”</i>	Tìm trong 10 file phụ đề .vtt của các buổi học các từ khóa (“bài tập lớn”, “đề án”, “deadline”...), xác định yêu cầu nằm ở Buổi 8, trích nội dung kèm mốc thời gian.
Dán bản tóm tắt yêu cầu em tự viết lại, hỏi <i>“như vậy đúng chưa?”</i>	Đối chiếu lại transcript, xác nhận các ý đúng, chỉ ra chỗ cần sửa: mốc thời gian cuối buổi, mục “source code” là suy luận do phụ đề tự động ghi sai, thiếu việc về nhà thứ hai (bài QHĐ trên WeCode).
<i>“...dựa vào những dữ liệu trên, hãy đưa ra những câu hỏi để giúp bạn có thể làm bài tập này.”</i>	Đặt 17 câu hỏi về: chọn bài, công nghệ, mức độ trực quan, report, video, hình thức nộp.
Trả lời 17 câu hỏi: dán đề bài “Đường đi an toàn” và hình minh họa, dán lời giải Python, chọn web (JS/Python), có deploy, minh họa từng bước cho người mới học, nhập tay + sinh ngẫu nhiên + validate đầy đủ, song ngữ, report LaTeX ghi trung thực, <i>“AI đề xuất rồi bạn kiểm chứng”</i> , AI tự làm video, làm cá nhân, nộp trong tuần.	Viết lời thuật toán JS; viết bộ test đối chiếu vết cạn (mục 4) và chạy; xây giao diện web; tự mở trình duyệt để kiểm tra (desktop, điện thoại, tiếng Anh, lưới 1000×1000 , input lỗi) và sửa các lỗi hiển thị phát hiện được; viết script quay video, phát hiện và khắc phục lỗi video bị hụt khung ở đáy; chụp ảnh và viết report này.
<i>“deploy github nhé, còn video lòng tiếng nghe kinh quá, có cách khác không”</i>	Tạo repo public trên GitHub và bật GitHub Pages; đề xuất 3 cách thay giọng đọc, em chọn giọng AI neural; AI chuyển script video sang edge-tts và tạo hai bản giọng nữ/nam để em chọn.

6.3 Phân định: phần nào do AI, phần nào do em

- **Chọn bài, yêu cầu sản phẩm** (đối tượng là người mới học, các chức năng cần có, song ngữ, hình thức report): em quyết định.
- **Thuật toán**: theo lựa chọn của em, *AI đề xuất – em kiểm chứng*. Công thức truy hồi, chứng minh và phân tích độ phức tạp ở mục 3 do AI trình bày; em kiểm chứng bằng chạy tay ví dụ trong đề, bằng bộ test đối chiếu vết cạn 307 trường hợp và test $n = 1000$ so với công thức tổ hợp $\binom{1998}{999}$.
- **Code giao diện, script test, script video, report**: AI viết, em đọc lại, chạy thử và chỉnh sửa.

[Cần sinh viên điền/xác nhận trước khi nộp – AI không biết các thông tin này]

- Nguồn gốc lời giải Python nộp WeCode (tự viết / tham khảo / dùng AI khác – nếu có)

thì ghi rõ công cụ và prompt): _____

- Có tra cứu Google/tài liệu nào không (từ khóa, trang tham khảo): _____
- Những chỉnh sửa em tự làm thêm sau khi AI tạo sản phẩm: _____

6.4 Nhận xét về việc dùng AI

- **Điểm mạnh:** AI làm rất nhanh phần “tay chân” (giao diện, kiểm thử, quay video) và tự kiểm tra sản phẩm trên trình duyệt, nhờ đó phát hiện được lỗi hiển thị (ô KTX bị che giá trị, khoảng trống thừa dưới lưới, tràn ngang trên điện thoại, video hụt khung).
- **Rủi ro:** AI có thể đưa ra lời giải sai nhưng trình bày rất thuyết phục. Vì vậy phần kiểm chứng độc lập (vét cạn, công thức tổ hợp) là bắt buộc chứ không chỉ tin vào lập luận.
- **Bài học:** Để dùng AI hiệu quả cần cung cấp đủ ngữ cảnh (đề bài, định dạng input, code hiện có, đối tượng người dùng). Việc AI hỏi lại 17 câu trước khi làm giúp sản phẩm đúng ý ngay từ đầu.

7. Đánh giá và hướng phát triển

Đã đạt: đủ ba sản phẩm nộp; giao diện có nhập liệu, kiểm tra input, minh họa từng bước, giải thích công thức, chạy được tới $n = 1000$; lời giải được kiểm chứng.

Hạn chế: hoạt hình từng ô chỉ áp dụng cho $n \leq 16$ để lưới còn đọc được; bản đồ nhiệt dùng số thực nên chỉ mang tính minh họa; giọng thuyết minh là giọng tổng hợp (AI) chứ không phải giọng thật.

Hướng phát triển: so sánh trực quan với cách vét cạn (đếm số lời gọi đệ quy) để thấy rõ lợi ích của QHĐ; mở rộng cho phép đi theo 3 hướng, hoặc tìm đường an toàn có chi phí nhỏ nhất.

A. Phụ lục: lời giải nộp WeCode (Python)

```

1  import sys
2
3
4  def solve():
5      input_line = sys.stdin.readline
6
7      line = input_line()
8      while not line.strip():
9          line = input_line()
10         n = int(line.strip())
11
12         MOD = 10**9 + 7
13         dp = [0] * n
14         dp[0] = 1
15
16         for _ in range(n):
17             row_line = input_line()
18             while not row_line.strip():
19                 row_line = input_line()
20             row = "".join(row_line.split())
21
22             for j in range(n):
23                 if row[j] == "*":

```

```
24     dp[j] = 0
25     elif j > 0:
26         val = dp[j] + dp[j - 1]
27         dp[j] = val if val < MOD else val - MOD
28
29     print(dp[n - 1])
30
31
32 if __name__ == "__main__":
33     solve()
```

B. Phụ lục: cấu trúc thư mục nộp

```
DoAn_DuongDiAnToan/
  app/index.html, app/core.js    giao dien web + loi thuat toan
  solution/safe_path.py         loi giai nop WeCode
  tests/run_tests.py            doi chieu vet can, test lon, test validate
  video/demo_*.mp4              video demo (co thuyet minh)
  video/make_video.py           script tao video tu dong
  report/report.tex, report.pdf bao cao nay
```